1MI 0800469 | Домашно

1.Валидиране (Counter Trick)

Вход: масив S[0..n-1] съдържащ само "(" и ")"

```
balance = 0                          O(1)
@2 for i = 0 to n - 1:               O(n-1)
    if S[i] == '(' then              O(1)
        balance = balance + 1        O(2)
    else
        balance = balance - 1        O(2)
    endif
    if balance < 0 then              O(1)
        return false                 O(1)
    endif
endfor
return balance == 0                  O(1)
```

1) Алгоритъма проверява дали масив от скоби има балансирани скоби. За всяка отворена скоба има след това една затваряща. Ако в даден момент затварящите са повече от отварящите, връща лъжа. Ако накрая има повече отварящи от затварящи, връща лъжа.

2) $I(k) \leftrightarrow$ преди k-тата итерация на @2 цикъла

$$balance_k = |\{i \mid A[i] = "("\ \text{за}\ i \in \{0..k-1\}\}| - |\{i \mid A[i] = ")"\ \text{за}\ i \in \{0..k-1\}\}|$$

$$balance_k \geq 0$$

3) $O(1) + O((n \cdot (1+2+2+1)+1)) + O(1) \in O(n)$

Времевата сложност $\in O(n)$

---

2. Two Sum (Two Pointers)

Вход: Сортиран масив A[0..n-1] от цели числа, target-цяло число.

```
L = 0                                O(1)
R = n - 1                            O(1)
@3 while L < R do                    O(n)
    sum = A[L] + A[R]                O(2)
    if sum == target then            O(1)
        return (L, R)                O(1)
    else if sum < target then        O(1)
```



```
        L = L + 1                    O(2)
    else
        R = R - 1                    O(2)
    endif
endwhile
return -1                            O(1)
```

1) Търси дали има двойка от цели числа в масив, , сумата им е равна на target

2) $I(k)$ – преди k-тата итерация на @3 while цикъла

Ако $\exists i, j$ т.е.

$$A[i] + A[j] = target$$

то $L_k \leq i < j \leq R_k$

3) Сложност: $O(1) + O(1) + O(n \cdot (2+1+1+2+2)+1) + O(1)$

$\in O(n)$

3.Мнозинство

Вход: Масив A[0..n-1] от естествени числа

```
candidate = null
count = 0
for i = 0 to n - 1 do
    if count == 0 then
        candidate = A[i]
        count = count + 1
    else if A[i] == candidate then
        count = count + 1
    else
        count = count - 1
    endif
endfor
return candidate
```

@3 $O(2)$, $O(n)$, $O(1)$, $O(1)$, $O(1)$, $O(1)$, $O(1)$, $O(1)$

1) Намира числото в $A[0..n-1]$, което се среща $\geq \lceil \frac{n}{2} \rceil$

2) $I(k)$ - k-тата итерация на цикъла на @3 ред

$candidate_k$ е единственият елемент, който може да е мнозинство в $P[0..k-1]$

$$count_k = |\{j \mid A[j] = candidate_k, j \in \{0..k-1\}\}| - |\{j \mid A[j] \neq candidate_k, j \in [0..k-1]\}|$$

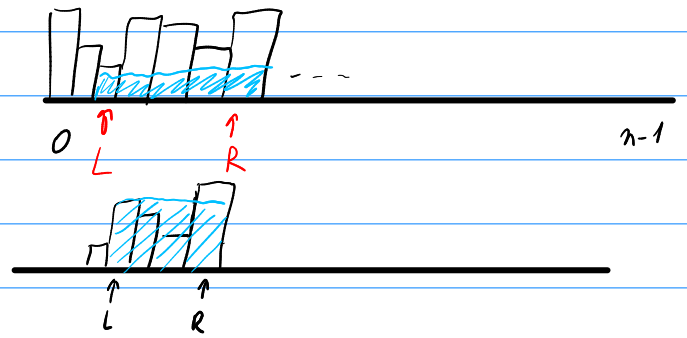
3) Понеже @3 while приключва след n стъпки $\Rightarrow$ Alг $\in O(n)$

4.Бидон с най-много вода

Вход: Масив H[0..n-1] от височини(>0).

```
L = 0                                   O(1)
R = n - 1                               O(1)
max_area = 0                            O(1)
while L < R do                          O(n)
    area = (R - L) * min(H[L], H[R])    O(4)
    max_area = max(max_area, area)      O(1)
    if H[L] < H[R] then                 O(1)
        L = L + 1                       O(1)
    else
        R = R - 1                       O(1)
    endif
endwhile
return max_area                         O(1)
```

@4



1) между n елемента в масив намира най-голямото число, което корespондира на вода, която би могла да се слети, без да прелива

2) $I(k)$: преди k-тата итерация на @4 цикъла

R_k и L_k са стойностите на R и L при k-тата итерация
$R_0 = n-1$ $L_0 = 0$

---

1) max_area е максималната стойност при $\forall$ преминати до МОМЕНТА двойки (R_i, L_j), $0 \leq i < k$, $k < j \leq n-1$

$I(k) \rightarrow$

2) Ако има по-оптимално решение то е в $[L_k, R_k]$

3) $O(1) + O(1) + O(1) + O(n(4+1+1+1+1)) + O(1) \in O(n)$

5.Алгоритъм на Кадан

Вход: Масив A[0..n-1] от цели числа

```
curr_max = 0
global_max = -100000
@3 for i = 0 to n - 1 do
    curr_max = max(A[i], curr_max + A[i])
    global_max = max(global_max, curr_max)
endfor
return global_max
```

1) Търси най-голямата сума на infix в $A[0..n-1]$

2) $I(k) \Leftrightarrow$ преди k-тата итерация на @3 while цикъла

$$curr_max_k = \max_{0 \le i \le k} \left\{ \sum_{j=i}^{k} A[j] \right\}$$

$$global_max_k = \max_{0 \le i \le j \le k} \left\{ \sum_{i}^{j} A[i] \right\}$$

3) На ред @3, for минава от 0, до $n-1$ за n стъпки
$\forall$ друга операция е const $\Rightarrow \in O(n)$

6.Заека и костенурката

Вход: Масив A[0..n] от естествени числа от 1 до n.

```
slow = A[0]                    O(1)
fast = A[A[0]]                 O(1)
@3 while slow != fast do       O(2n)
    slow = A[slow]
    fast = A[A[fast]]
endwhile
slow = 0
@8 while slow != fast do       O(n)
    slow = A[slow]
    fast = A[fast]
endwhile
return slow
```

1) Търси се цикъла в $A[0..n]$
(A е представяне на входа на графа linked list)

2) $I_1(k) \Leftrightarrow$ k-итерация на @3 while

slow $= k$ fast $= 2.k$

(Ще използваме цикличност и твърд, че заека достига кост. след $q+1$ стъпки, където q е разст. от заека до кост., след влизането в цикъла)

$I_2(i) \Leftrightarrow$ slow и fast са на еднакво разстояние от началото на цикъла

3) while на @3 ще отнеме $O(m + k.c)$, където m е дължин. на пътя до преди цикъла, k е дълж. на цикъла, а c е const

while на @8 ще отнеме $O(m) \Rightarrow$ Алг ще отнеме $O(2.m + p)$ $\rightarrow p = k.c$ или по-точно, след колко стъпки ще се заекат заека и кост в цикъла
или $O(n)$

[Най-лошия случай: заека преди кост. при влизане в цикъла и между тях има $k-2$ стъпки $\Rightarrow$ след $k-1$ стъпки, заека ще стигне кост.
$\Rightarrow O(m + k - 1)$ или $O(n)$]

7. Exponentiation by Squarin

Вход: x естествено число,n естествено число.

```
res = 1                                  O(1)
base = x                                 O(1)
power = n                                O(1)
while power > 0 do                       O(p)
    if power mod 2 == 1 then             O(2)
        res = res * base                 O(2)
    endif
    base = base * base                   O(2)
    power = floor(power / 2)             O(2)
endwhile
return res                               O(1)
```

@3

1) Алгоритъм приема две числа $x \in \mathbb{N}$ и $n \in \mathbb{N}$ и връща x^n (x на степен n)

2) $I(k) \leftrightarrow$ при k-тата итерация на цикъла по ред@3

$\hookrightarrow res_k * base_k^{power_k} = x^n$

$base_k = base_0^{(k+1)}$

$power_k = \left\lfloor \frac{power_k}{2^k} \right\rfloor$

$res_k = \begin{cases} res_{k-1} * base_{k-1}, & k \pmod 2 = 1 \\ res_{(k-1)}, & k \pmod 2 = 0 \end{cases}$

не се изпълнят по време на подразбиране

3) $\underbrace{O(3)}_{init} + O(\underbrace{p}_{while}(\underbrace{\cdot\ 2+2+2+2}_{do\ \dots\ endwhile})) + \underbrace{O(1)}_{return}$

k	0	1	2	—	k
p	n	$\frac{n}{2}$	$\frac{n}{4}$	—	$\frac{n}{2^k}$

$\frac{n}{2^k} = 1$ $\quad n = 2^k$

$\log_2 n = \log_2 2^k$

$\log_2 n = k$

$= O(p) = O(\log_2 n)$

$\Rightarrow O(3) + O((\log_2 n) \cdot (8)) + O(1) \in \underline{O(\log_2 n)}$

Ал. $\in O(\log n)$

8. Знаме (3-Way Partition)

Вход: Масив A[0..n-1] съдържащ само 0,1 и 2.

```
L = 0
mid = 0
R = n - 1
@4 while mid <= R do
    if A[mid] == 0 then
        swap(A[L], A[mid])
        L++, mid++
    else if A[mid] == 1 then
        mid++
    else
        swap(A[mid], A[R])
        R--
    endif
endwhile
return A[0..n-1]
```

1) Масива го връщаме сортиран

3) mid <= R, mid започва от нула, а R от n-1 при всяка итерация или или mid се добавя 1 или от R се вади 1. => След краен брой стъпки ще получим m > R и ще приключим

=> O(n)

2) I(k) <-> На k-итерация на @4 while

$A[0..L_k-1] = [0...0]$

$A[L_k..mid_k-1] = [1...1]$

$A[mid_k..R_k]$ случайни / неразгледани

$A[R_k+1..n-1] = [2...2]$

9.Лексикографска пермутация(Ама каква?)

Вход: Масив A[0..n-1] от естествени числа

```
    i = n - 2
@2  while i >= 0 and A[i] >= A[i+1] do      O(n)
        i--
    endwhile
    if i >= 0 then
        j = n - 1
@7      while A[j] <= A[i] do               O(n)
            j--
        endwhile
        swap(A[i], A[j])
    endif
    reverse(A, i + 1, n - 1)                O(n)
```

1) Алгоритъма преобразява масива в следващата по-голямата пермутация.

Тоест ако пуснем A в алгоритъма и върне A'

$$\sum_{i=0}^{n-1} A[i] * 10^i < \sum_{i=0}^{n-1} A'[i] * 10^i$$

Това важи освен ако $A[i] \geq A[i+1]$, за $i \in \{0...n-2\}$, тогава ще върне най-малката пермутация

3) Понеже в A има n елемента, @2 се движи от n-2 до 0 и ще приключи в O(n) време

Аналог за @7. Ако i=0 и $A[i] >= A[p]$ за $\forall p \in \{0...n-1\}$, то ще отнеме n стъпки или @7 ще прод. O(n) време.

reverse е O(n) => Алг е $\in O(n)$

$i = n-2-k+1$

2) $I_1(k)$ - k-тата итерация на @2 while - $A[i...n-1]$ намаляващи елементи $a_i \geq a_{i+1} \geq ... \geq a_{n-1}$

$I_2(k)$ - k-тата —||— @7 —||— $a \in A[j+1...n-1]$

$a \leq A[i]$ $j = n-k$

10. Trapping Rain Water (Two Pointers)

Вход: Масив H[0..n-1] от височини(>0).

```
L = 0
R = n - 1
max_L = 0
max_R = 0
water = 0
@6 while L < R do
    if H[L] < H[R] then
        max_L = max(max_L, H[L])
        water = water + (max_L - H[L])
        L++
    else
        max_R = max(max_R, H[R])
        water = water + (max_R - H[R])
        R--
    endif
endwhile
return water
```

$\}\ O(5)$

$O(n)$

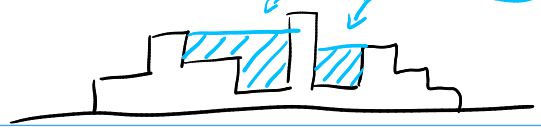
$O(1)$

$\}\ O(3)$

$\}\ O(3)$

$O(1)$

1) Вряща възможната „задържана вода"



2) $I(k) \leftrightarrow$ k-та итерация на @6 while

$$water_k = \sum_{i=0}^{L_k-1} \left(\max\{A[0..i]\} - H[i] \right) + \sum_{j=R_k+1}^{n-1} \left(\max\{A[j..n-1]\} - H[j] \right)$$

3) While работи докато $L < R$. В началото $L=0, R=n-1$ и на всяка итерация $L+=1$ или $R-=1$ => след n стъпки спира => $\in O(n)$

11.Hoare's Selection

Вход: Масив A[0..n-1] от естествени числа, естествено число k.

```
low = 0
high = n - 1
@3 while low <= high do
    pivot_idx = partition(A, low, high)
    if pivot_idx == k then
        return A[k]
    else if pivot_idx < k then
        low = pivot_idx + 1
    else
        high = pivot_idx - 1
    endif
endwhile
```

$\}\ O(1)$

$O(n)$

1) намиране k-тия елемент в масив

2) $I(i) \leftrightarrow$ i-тата итерация на @3 while

k-тия елемент е в $A[low_i, high_i]$

$low_i \le high_i$

3) partition сортира $O(n)$: Ако масива е в обратен ред [6,5,4,3,2,1] ще стане [1,6,5,4,3,2]. Ако търсим 6 елемент ще намерим само първия low = 1+1 = 2, и т.н. => partition $\in O(n)$ и while $\in O(n)$

Такъв ще отнеме $O(n*n)$ итерации в най-лошия случай

=> $\in O(n^2)$

12. Knuth-Morris-Pratt (LPS Array Construction)

Вход: string P[0..m-1].


```
lps = array of size m
lps[0] = 0
len = 0
i = 1
@4 while i < m:
    if P[i] == P[len] then
        len++
        lps[i] = len
        i++
    else
        if len != 0 then
            len = lps[len - 1]
        else
            lps[i] = 0
            i++
        endif
    endif
return lps
```

1) За даден масив P ще върне lps, където за подмасив P[0..i], lps[i] ще ни каже макс дължина на prefix, който е suffix

2) $I(k) \leftrightarrow$ k-тата итерация на @4

$len_k = \max\limits_{0 \le q \le j \le i_k} |P[0..q]|$ т.че $P[0..q] = P[j-q \dots j]$ и $q < j-q$

$lps[j] = \max\{q \mid 0 \le q < j,\ P[0..q-1] = P[j-q+1 \dots j]\}$

$j \in \{0 \dots k-1\}$

3) На @4 while, $i < m$ и i се движи напред с 1 стъпка в $\frac{2}{3}$ случая.

В $\frac{1}{3}$ случая len намалява, но крайно, понеже $lps[0] = 0$, => Алг е $O(m)$

13. Медиана

Вход: Масив A[0..m-1] и масив B[0..n-1] от естествени числа , $m \le n$.

```
low = 0
high = m
half = (m + n + 1) / 2
@4 while low <= high do
    i = (low + high) / 2
    j = half - i
    maxLeftA = (i == 0) ? -inf : A[i-1]
    minRightA = (i == m) ? inf : A[i]
    maxLeftB = (j == 0) ? -inf : B[j-1]
    minRightB = (j == n) ? inf : B[j]

    if maxLeftA <= minRightB and maxLeftB <= minRightA then
        return max(maxLeftA, maxLeftB)
    else if maxLeftA > minRightB then
(*)     high = i - 1
    else
        low = i + 1
    endif
endwhile
```

1) Намира медианата на 2 масива дължини m и n, ако е един сортиран (алгоритъма е приложим да се използва съединение и сортиране на масиви)

3) Полагаме low = 0 high = m, $i = (low + high)/2$

В края на цикъла, или връщаме резултат или променяме low и high след i

Но понеже i ще е напрактика средата след ∀ изпълнения на (*) редовете, се махат $\frac{1}{2}$ от елем. => Алг. $\in O(\log m)$

2) $I(k) \Leftrightarrow$ за k-тата итерация @ 4

$\exists i$ т.че $low_k \le i \le high_k$ и

$$\max LeftA_k \le \min RightB_k$$
$$\max LeftB_k \le \min RightA_k$$

и медианата е $\max(\max LeftA_k, \max LeftB_k)$

Домашно - Invariants (група 1, 2, 3) ДАА

от: Цветомир Спайков, КН, 3 курс, 2 група

1MI0800469

22.03.2026г.